

SymPy Bug Report

1 Bug 17: Telescoping Sum Ignores Undefined Terms

1.1 Minimal Reproducer

```
from sympy import Sum, oo, symbols

k = symbols("k", integer=True)
s = Sum(1/(k*(k - 1)), (k, 0, oo))

print("actual_sum =", s.doit())
print("first_terms =", [s.function.subs(k, i) for i in range(5)])
print("tail_sum_from_2 =", Sum(1/(k*(k - 1)), (k, 2, oo)).doit())
```

1.2 Observed Behavior

```
actual_sum = -1
first_terms = [zoo, zoo, 1/2, 1/6, 1/12]
tail_sum_from_2 = 1
```

SymPy returns the finite value -1 for a series whose first two terms are undefined. The output is not merely an alternative closed form for the same mathematical object: the requested series includes poles at the start of the summation range.

1.3 Expected Behavior

The call should not evaluate to the finite value -1 . The summand is undefined at $k = 0$ and $k = 1$, so the infinite sum from $k = 0$ is undefined in the ordinary sense. A correct result should preserve that undefinedness, for example by leaving the sum unevaluated or returning an explicit non-finite/undefined value rather than a finite number.

1.4 Mathematical Explanation

The summand has the partial-fraction form

$$\frac{1}{k(k-1)} = \frac{1}{k-1} - \frac{1}{k},$$

which telescopes only on ranges where all involved terms are defined. On the well-defined tail,

$$\sum_{k=2}^N \frac{1}{k(k-1)} = \sum_{k=2}^N \left(\frac{1}{k-1} - \frac{1}{k} \right) = 1 - \frac{1}{N},$$

and therefore

$$\sum_{k=2}^{\infty} \frac{1}{k(k-1)} = 1.$$

The requested sum, however, starts at $k = 0$. Its first two terms are $1/(0 \cdot -1)$ and $1/(1 \cdot 0)$, both undefined. Telescoping cannot cancel away terms that do not exist, so the finite value -1 is mathematically invalid.

1.5 Source-Level Diagnosis

The diagnosis identifies the faulty path in `sympy/concrete/summations.py`. The public call `Sum(...).doit()` reaches `Sum.doit`, which calls `eval_sum`. That then calls `eval_sum_symbolic`, which applies `apart`, splits the resulting addends, and invokes `telescopic` around `sympy/concrete/summations.py:1163-1171`.

For this summand, partial fractions produce $1/(k-1) - 1/k$. The telescoping logic recognizes $-1/k$ as a shifted negative copy of $1/(k-1)$ and accepts the telescoping endpoint formula. Endpoint construction in `telescopic_direct`, identified at `sympy/concrete/summations.py:946-967` (endpoint `Add` at line 967), returns an expression equivalent to evaluating L at the lower endpoint and R at infinity. For limits $(k, 0, \infty)$, this yields $-1 + 0 = -1$.

The source-level problem is that this shortcut assumes the summand and the decomposed telescoping terms are finite over the summation range. In this case, the original summand has poles at $k = 0$ and $k = 1$, and the partial-fraction terms expose exactly the singular pieces that the endpoint formula skips. The suggested fix direction in the diagnosis is to check for poles of the original summand in the summation range, especially finite initial terms of infinite sums, before accepting a telescoping result. If such singular terms are present, SymPy should avoid producing a finite telescoping value.

1.6 Additional Failing Instances

The independent verification exercises the shifted family

$$\sum_{k=s}^{\infty} \frac{1}{(k-s)(k-s-1)}$$

for $s \in \{0, 1, 2, 3, 4, 5\}$. In every case, the first two terms occur at $k = s$ and $k = s + 1$, where the denominator is zero. The well-defined tail from $k = s + 2$ has the same telescoping value 1, but the full requested series is undefined because it includes the singular initial terms.

Input / Expression	SymPy behavior	Expected behavior
$\sum_{k=0}^{\infty} \frac{1}{k(k-1)}$	returns finite value -1	undefined; poles at $k = 0, 1$
$\sum_{k=1}^{\infty} \frac{1}{(k-1)(k-2)}$	returns finite value -1	undefined; poles at $k = 1, 2$
$\sum_{k=2}^{\infty} \frac{1}{(k-2)(k-3)}$	returns finite value -1	undefined; poles at $k = 2, 3$
$\sum_{k=3}^{\infty} \frac{1}{(k-3)(k-4)}$	returns finite value -1	undefined; poles at $k = 3, 4$
$\sum_{k=4}^{\infty} \frac{1}{(k-4)(k-5)}$	returns finite value -1	undefined; poles at $k = 4, 5$
$\sum_{k=5}^{\infty} \frac{1}{(k-5)(k-6)}$	returns finite value -1	undefined; poles at $k = 5, 6$

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The closest public material found was SymPy documentation for telescoping and Gosper-style summation machinery, along with unrelated or different summation issues such as GitHub issues 12018 and 5229. The deduplication verdict is that this is a specific new instance of a known failure family: open SymPy issue [#8746](#) (“summation over poles of rational functions”) already documents the same failure mode of summation across poles returning a finite closed form, but this exact instance where `Sum.doit()` ignores singular initial terms of an infinite sum and returns a finite telescoping value is not separately reported. This remains individually actionable because the reproducer is concise, the affected summation path is identified, and the proposed regression test directly protects the problematic behavior.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Sum, oo, symbols

@pytest.mark.parametrize("shift", [0, 1, 2, 3, 4, 5])
def test_telescoping_sum_does_not_ignore_undefined_initial_terms(shift):
    k = symbols("k", integer=True)
    expr = 1/((k - shift)*(k - shift - 1))
    result = Sum(expr, (k, shift, oo)).doit()
    assert result.is_finite is not True
```